UNIVERSITÀ DI PISA

DATA MINING AND MACHINE LEARNING PROJECT

COMPARING STOCK PRICE AND TWEETS

Artificial Intelligence and Data Engineering

Emilio Paolini 544485

Abstract

The aim of this project is to understand if it is possible to predict the stock price using the public opinions. In particular, we have examined the case of Apple stock price and using the tweets related to the Apple topic, we have tried to realize if there exists a correlation between the sentiment contained in the tweets and the trend in the stock market. The tweets were fetched using particular criteria (set of keywords contained in the text of the tweets). Then they have been processed using textual elaboration techniques and finally a classifier was obtained to extract the sentiment contained in the tweets. Specifically, each tweet has been classified as "positive", "negative" or "neutral". We tested many classifiers, and compare the results of each of them using typical classifier comparison techniques. The best results were obtained by the Naive Bayes Multinomial classifier. Then applying this classifier to daily tweets and comparing the sentiment contained in them to the stock price should give us a solution to our problem: is it possible to predict the stock price using sentiment in tweets? Unfortunately, the answer is no, it's not possible.

Contents

Introduction 4

Retrieving tweets 5

Text elaboration 5

The Supervised Learning Stage 7

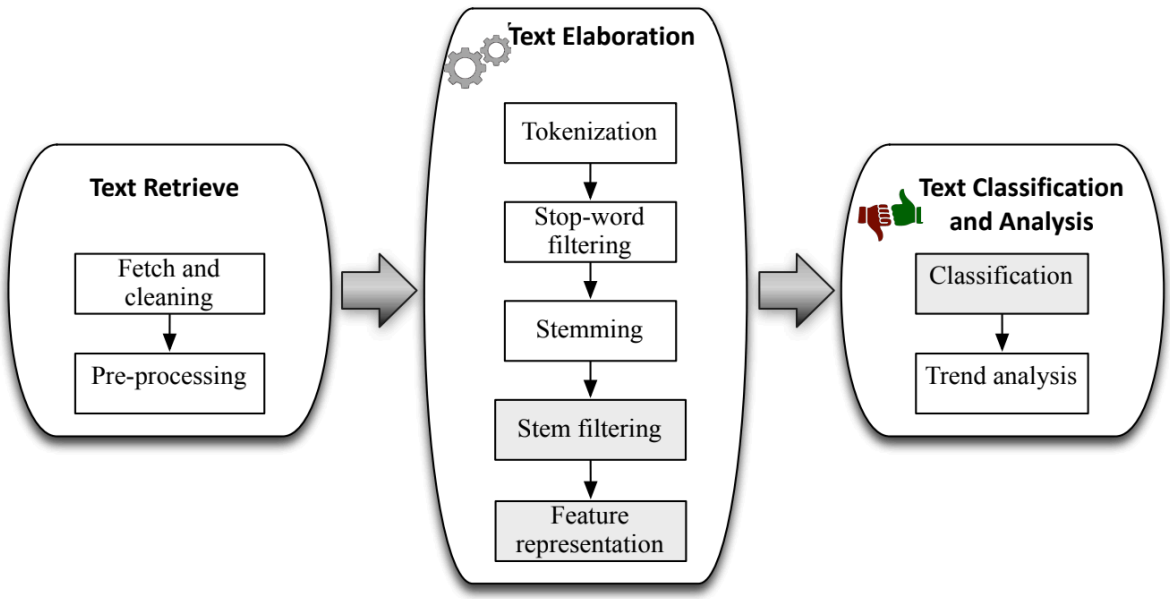
Classifiers comparison 7

The application 9

Conclusion 13

Introduction

Sentiment analysis, the task of mining subjective feelings expressed in the text, has been found to play a significant role in many applications such as product recommendations, healthcare, politics, and in surveillance. This project tries to use the sentiment analysis to solve a very interesting challenge regarding the finance: the prediction of the stock price trend. The idea was to obtain the data from some social networks, analyze them and compare them with the stock price. Among all the existing social networks, the best suitable for this application was Twitter, used mainly to express opinions about topics. In this social network, users typically express their thoughts about a certain topic, product, service, event, etc., using a public Status Update Message called **tweet**. A tweet may also contain meta-information such as links, hashtags, emoticons, images. Analyzing tweets is a very complex task and requires a lot of steps to be performed. In the following sections, we will present a method to collect and clean tweets from useless information, use them to train various classifiers and choose the best one according to our context. The tasks that have been performed are summarized in the following image:



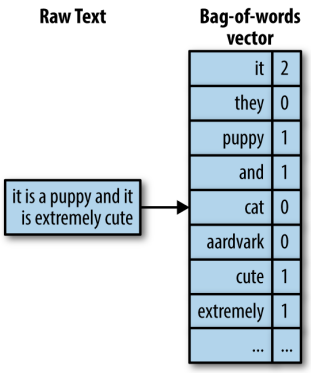
Retrieving tweets

The first step to perform is to download tweets related to Apple topic by day. In this step tweets are fetched using some criteria (keywords and date of posting). Twitter offers official APIs to retrieve tweets by using some parameters, but they are limited in the free version (in fact it is possible to retrieve tweets until 7 days in the past). To bypass this limitation the Java Library **GetOldTweets** was used. After downloading tweets and storing them in separate files (each file for each day of which we have stock price data), we have to apply a preprocessing step. This is because tweets have useless meta-information for the classification task. In fact, a downloaded tweet may contain mentions of other users, links, images, special characters. So we applied regular expression to remove all these useless information. Hashtags have received a different treatment: they were not completely discarded but reduced to words (by removing the symbol "#"). This is because an hashtag can also express sentiment. This is confirmed by considering a simple hashtag as #good that contains sentiment.

Text elaboration

The main aim of this step is to transform the cleaned tweets into a set of numeric vectors, by eliminating noise and extracting useful information. The text elaboration is composed by several tasks, that must be performed in the correct order, otherwise the entire process doesn't make any sense. The order of the tasks is the following:

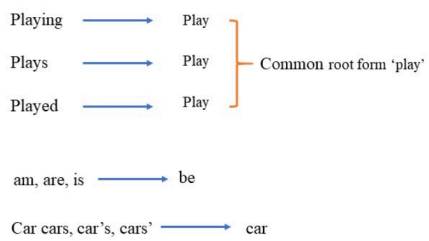
1 - **Tokenization**: this operation consists of transforming a stream of characters into a stream of processing units called **tokens**. After this step, each tweet is represented as a set of words, according to the Bag-Of-Word representation:



In particular, the tokenizer that has been used is the **N-Gram tokenizer**, which first breaks text down into words whenever it encounters one of a list of specified characters, then it emits N-grams of each word of the specified length. N-grams are like a sliding window that moves across the word - a continuous sequence of characters of the specified length.

2 - **Stop-word filtering**: as the name suggests, this operation consists of filtering the stop-words. These words are generally the most common words in a language and they don't contain useful information so they can be considered as noise. Typically, they are articles, conjunctions, prepositions and pronouns.

3 - **Stemming**: this operation consists of reducing each word to its **stem** or **root form**, by removing its suffix, in order to group words having closely related semantics:



After this step, each tweet is represented as a sequence of stems. In particular, the stemmer that has been used is the **Snowball Tartarus Stemmer**.

4 - **Stem filtering**: during this operation the number of stems are reduced, by removing noisy stems and maintaining only the most relevant ones.

5 - **Feature representation**: this operation consists of building for each text the corresponding vector of numeric features, in order to represent all the texts in the same F-dimensional feature space. So each tweet is associated with a vector of binary or numeric features.

The Supervised Learning Stage

In this step we need to perform the **stem filtering** by identifying the most relevant stems, the **feature representation**, and **classification**. To do all these tasks we need a set of labelled tweets as **training set**. The dataset was retrieved from the following page https://www.kaggle.com/c/apple-computers-twitter-sentiment2/data, that contains about 1890 labelled tweets (negative, positive and neutral) about Apple. First of all the entire training set was cleaned using the preprocessing techniques explained before. Then each tweet of the training was sent through the following text mining steps: tokenization, stop-words filtering and stemming. The complete set of stems is generated by putting together all the stems extracted from the set of training text after the stemming step. To reduce the number of relevant stems we applied an **attribute selection filter**, that consists of two different types of algorithms:

- **evaluator** that determines the merit of single features or subsets of features. In particular, as evaluator we have used the **information gain**.
- **search algorithm** that proposes new feature subsets. As for the search algorithm, the **ranker** has been used.

Classifiers comparison

The classifiers that have been tested are three:

- **Support Vector Machine (SVM)**: the objective of the support vector machine algorithm is to identify a hyperplane in an N-dimensional space (where N is the number of features) that distinctly classifies the data points. To separate the two classes of data points, there are many possible hyperplanes that could be chosen. The objective is to identify a plane that has the maximum distance between data points of

both classes. Maximizing the margin distance provides some reinforcement so that future data points can be classified with more confidence.

- **Naive Bayes**

- **Naive Bayes Multinomial**

Then for each of them, we have also tested if applying the bagging and the boosting really improves the performance. The results of the tests are shown in the following table:

CLASSIFIER	CLASS	F-MEASURE	RECALL	PRECISION	AUC	ACCURACY	CONFUSION MATRIX
SVM	*positive*	0.420	0.303	0.688	0.687	72.92%	a b c <-- classified as
	neutral	0.799	0.886	0.728	0.734		66 33 119 | a = positive
	negative	0.668	0.609	0.740	0.773		6 370 232 | b = negative
		0.713	0.729	0.727	0.741		24 97 940 | c = neutral
NAIVE BAYES	*positive*	0.356	0.307	0.424	0.716	50.13%	a b c <-- classified as
	neutral	0.443	0.301	0.839	0.801		67 127 24 | a = positive
	negative	0.572	0.921	0.415	0.817		11 560 37 | b = negative
		0.475	0.501	0.655	0.796		80 662 319 | c = neutral
NAIVE BAYES MULTINOMIAL	*positive*	0.447	0.408	0.494	0.765	70.85%	a b c <-- classified as
	neutral	0.760	0.713	0.815	0.817		89 58 71 | a = positive
	negative	0.709	0.809	0.632	0.863		15 492 101 | b = negative
		0.708	0.709	0.719	0.826		76 229 756 | c = neutral
SVM + BOOSTING	*positive*	0.429	0.312	0.687	0.674	72.71%	a b c <-- classified as
	neutral	0.797	0.871	0.735	0.757		68 38 112 | a = positive
	negative	0.667	0.625	0.716	0.785		7 380 221 | b = negative
		0.713	0.727	0.723	0.756		24 113 924 | c = neutral
NAIVE BAYES + BOOSTING	*positive*	0.386	0.294	0.561	0.647	55.43%	a b c <-- classified as
	neutral	0.544	0.403	0.836	0.700		64 119 35 | a = positive
	negative	0.593	0.911	0.439	0.718		5 554 49 | b = negative
		0.542	0.554	0.676	0.699		45 588 428 | c = neutral
NAIVE BAYES MULTINOMIAL + BOOSTING	*positive*	0.454	0.417	0.497	0.704	69.79%	a b c <-- classified as
	neutral	0.750	0.713	0.791	0.775		91 50 77 | a = positive
	negative	0.693	0.773	0.628	0.821		15 470 123 | b = negative
		0.697	0.698	0.705	0.781		77 228 756 | c = neutral
SVM + BAGGING	*positive*	0.403	0.271	0.787	0.692	73.13%	a b c <-- classified as
	neutral	0.805	0.907	0.724	0.793		59 36 123 | a = positive
	negative	0.658	0.590	0.742	0.831		6 359 243 | b = negative
		0.711	0.731	0.737	0.794		10 89 962 | c = neutral
NAIVE BAYES + BAGGING	*positive*	0.369	0.298	0.485	0.714	50.98%	a b c <-- classified as
	neutral	0.459	0.315	0.850	0.798		65 131 22 | a = positive
	negative	0.572	0.926	0.414	0.815		8 563 37 | b = negative
		0.485	0.510	0.667	0.794		61 666 334 | c = neutral
NAIVE BAYES MULTINOMIAL + BAGGING	*positive*	0.418	0.376	0.471	0.765	70.11%	a b c <-- classified as
	neutral	0.756	0.713	0.804	0.824		82 58 78 | a = positive
	negative	0.701	0.796	0.627	0.865		18 484 106 | b = negative
		0.700	0.701	0.709	0.830		74 230 757 | c = neutral

How can we compare these classifiers using these indices? First of all we must determine what is more important for us. The precision can be a good parameter to compare different classifiers in case of balanced dataset, but in our case it cannot be applied directly because of the distribution of the classes in the training set (that is also used to test the classifiers using the 10-fold cross validation): in fact the number of positive tweets is 218, the number of negative tweets is 608 and the number of neutral tweets is 1061. If we would implement a classifier that always classifies instances as "neutral" we would obtain an accuracy of 56%. So the accuracy cannot really be used directly. Typical measures that we would like to use are **F-measure** (that we can consider as a summarization of the **recall** and the **precision**, since that it is defined as (2 * Precision * Recall) / (Precision + Recall)) and **AUC-Area under the curve**. Since that for us all the classes are important, we should consider the values on the fourth rows for each classifier, that correspond on the weighted average of the corresponding values. So the best classifiers in terms of F-measure are the SVM, the SVM+Boosting, the SVM+Bagging, the Naive Bayes Multinomial and the Naive Bayes Multinomial+Bagging. The versions of SVM that use boosting and bagging may not be considered since they don't **really** improve the performances and are more complex (in fact when we have to choose a classifier over the others we should also consider the complexity of it). Same reasoning can be applied to the Naive Bayes Multinomial+Bagging. So at the end we can conclude that both SVM and Naive Bayes Multinomial are similar. If we would want to go further and really choose one over the other, we should consider the AUC values. The Naive Bayes Multinomial has an higher value of this index than the SVM and so can be considered a better classifier for this application.

The application

All the classifiers seen before have been implemented inside a Java application that retrieves both tweets and stock price data about Apple using Alpha Vantage API. In particular the tweets retrieved where the ones containing the following words: #AAPL,

AAPL, @apple, #Apple, iPhone, Airpods, Apple Watch. These words are the most popular words related to Apple topic.

The main actor of the application is a generic user, who will use the functionalities offered by the software.

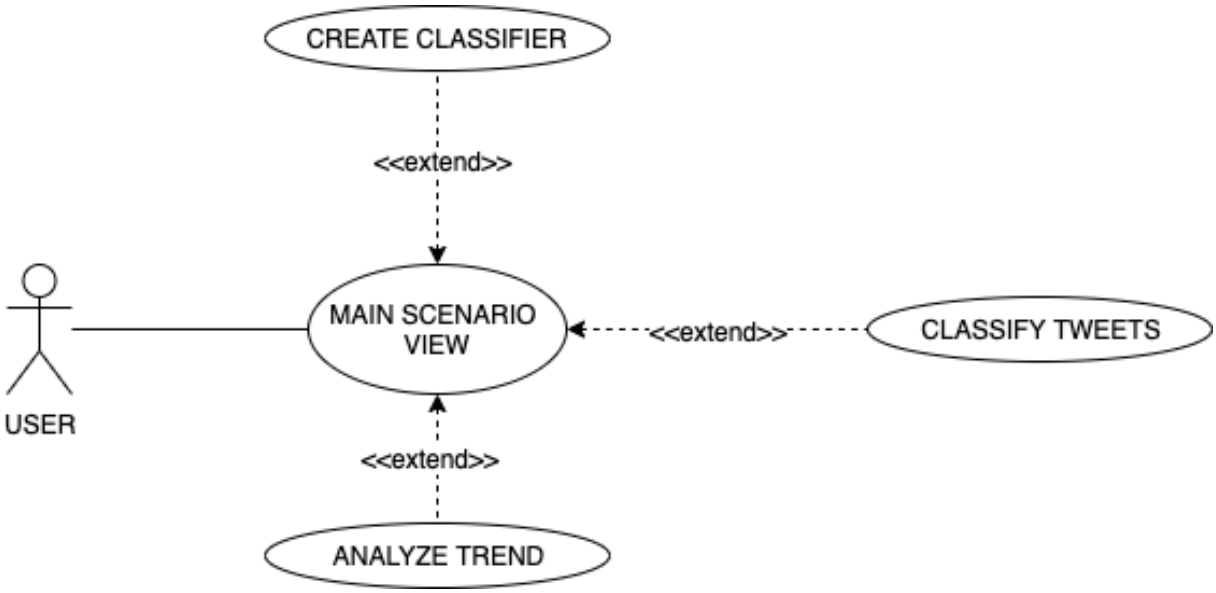
The application has to allow the users to do the following actions:

1- to **create** a classifier

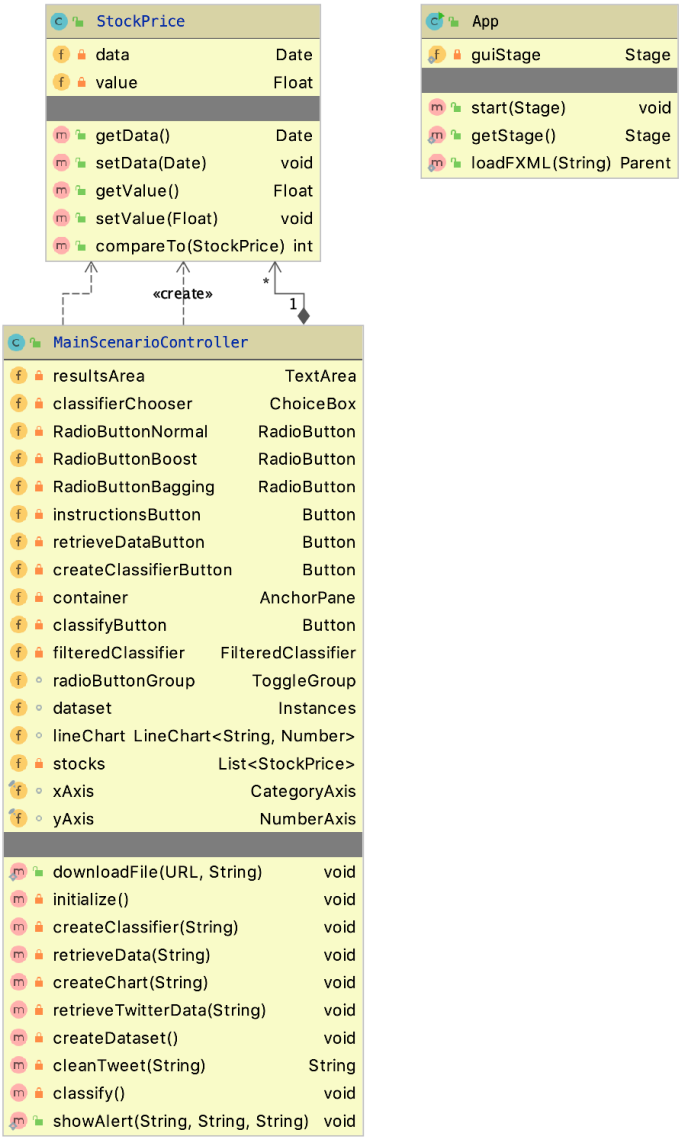
2- to **classify tweets** using the classifier created

3- to **analyze** the trend of the stock price related to tweets

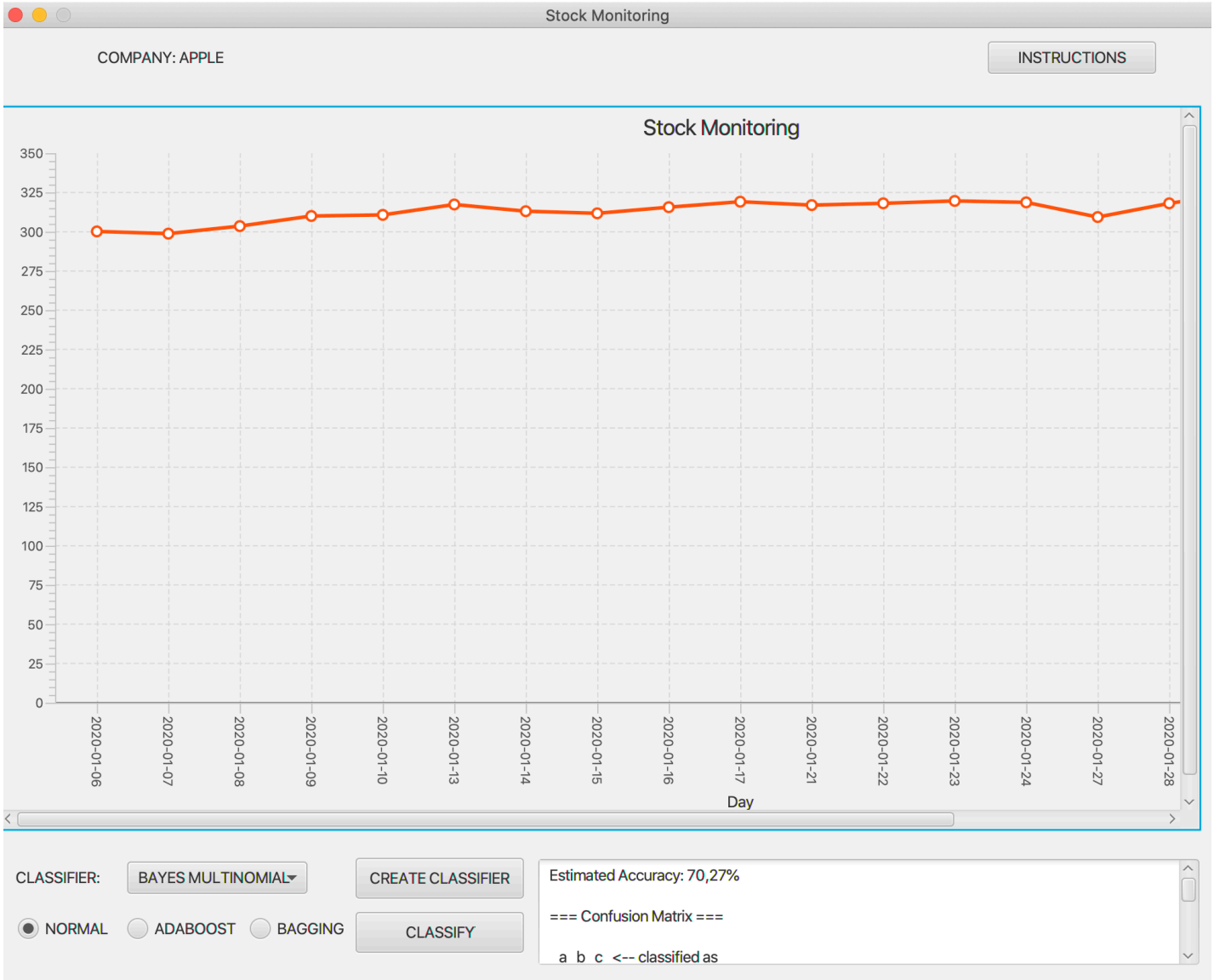
The application has only one scenario where there are implemented all the functions that it offers. The scenario is called **Main Scenario**.



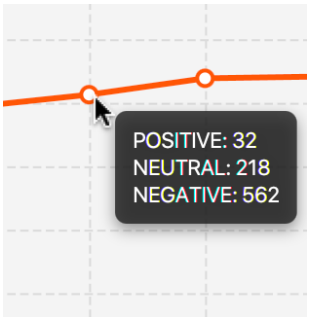
The UML of the application is the following:



When the application starts, two popups appears: the first one informs the user that the classifier has been created (by default the application uses the Naive Bayes Multinomial classifier) and the second one informs the user that the classification has been done. Then the application window appears as the following:



Each point in the line chart corresponds to the closing stock price of corresponding day. If the mouse is over a point, then a popup appears to inform the user about the tweets of that day. This situation is shown in the following image:



Then, in the bottom part, the user can also choose to adopt one of the classifiers mentioned before, including the possibility to apply boosting or bagging. Once the parameters are set, the user has to click on the "CREATE CLASSIFIER" button. A popup will appear to inform that the classifier has been created and the results of the 10-fold cross validation will appear in the the text area. If the user wants to classify the tweets using the new classifier create, he/she has to click on the "CLASSIFY" button. Again, a popup will appear to inform when the operation is completed.

Conclusion

Thanks to this application, we can understand if the changes in the trend of the stock price are really related to the public opinion expressed on Twitter. If there exists some correlation between the two things, one would expect to have a lot of negative classified tweets when there is a loss in stock price. In parallel, one would expect an increase in the number of positive classified tweets when there is a gain in stock price. These are just expectations and by monitoring the sentiment analysis on tweets and the stock price these expectations aren't matched. Why? There can be many factors that can play very important roles in the stock market. And the public opinion expressed in tweets about a company doesn't have any impact on the investors. It is also true that it's more common to express a negative sentiment when talking about a company instead of a positive comment. So for all these reasons we can't really use tweets for monitoring the stock price of a company and so it's also impossible trying to predict changes in trends.